

CLiPE

Централизованная система управления доступом
для Linux-хостов



Выполнил:
Студен группы 3093
Михайлов Роман

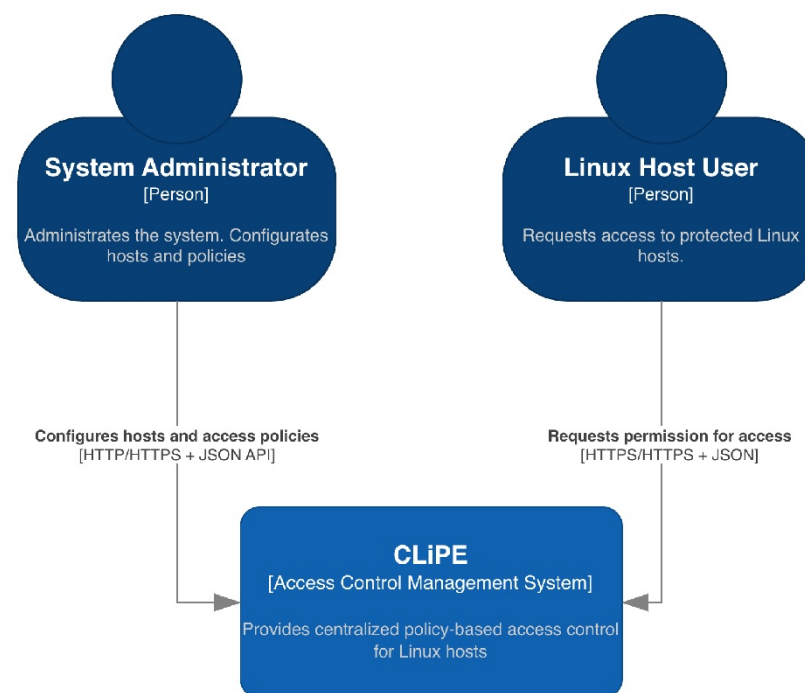
Постановка задачи

Цель

Создание централизованной системы управления доступом для Linux-хостов на основе PAM и политик доступа.

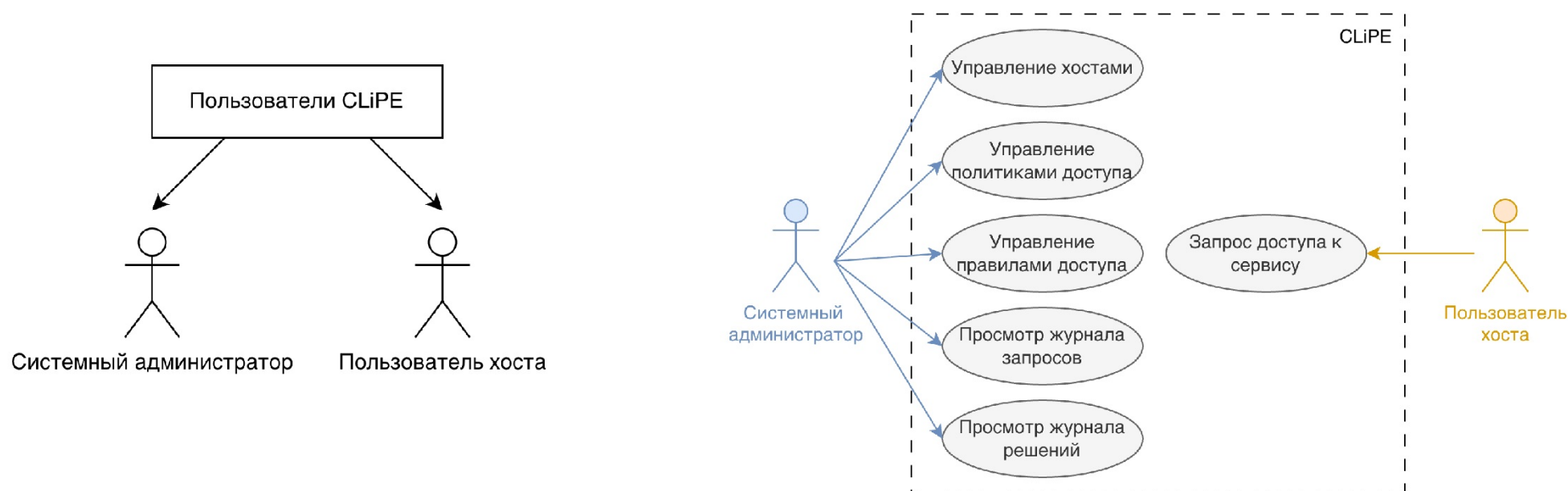
Предлагаемое решение:

- разделение компонентов на точку инициирования проверки и сервер принятия решений;
- централизованное хранение и управление политиками доступа;
- принятие решений на основе пользователя, сервиса, хоста, временного контекста и других атрибутов;
- журналирование запросов и решений.



Проектирование субъектов взаимодействия

Предусмотрено две роли для взаимодействия с системой: системный администратор и пользователь Linux-хоста, запрашивающий доступ к сервису

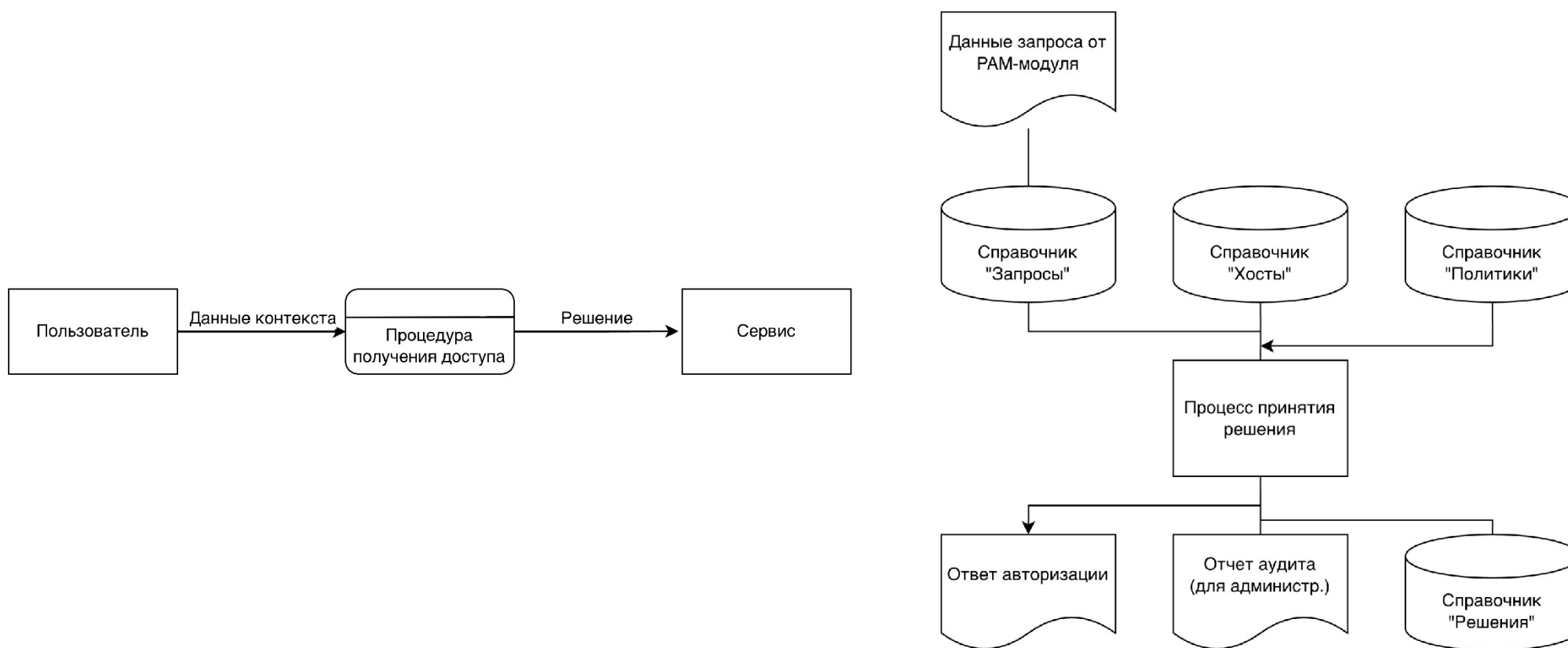


Проектирование функциональности

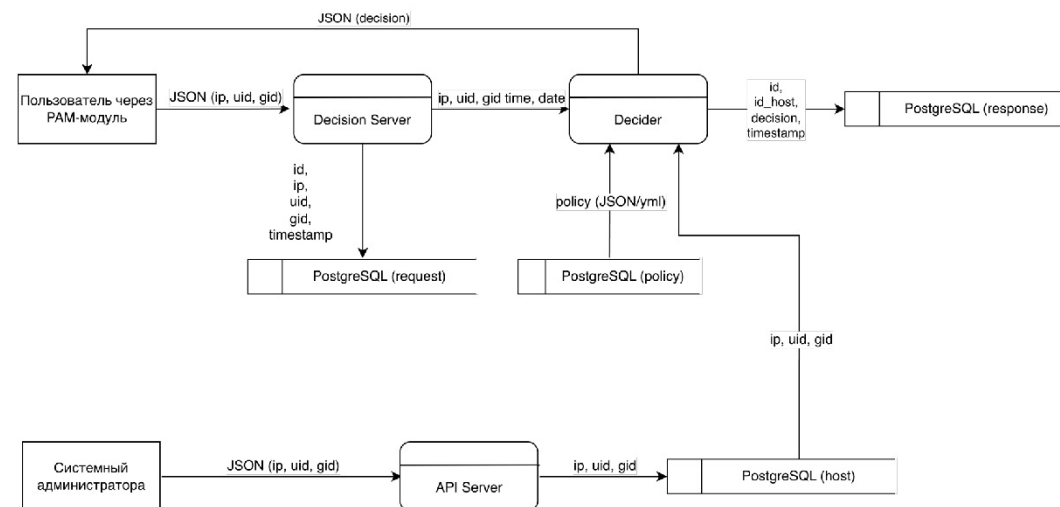
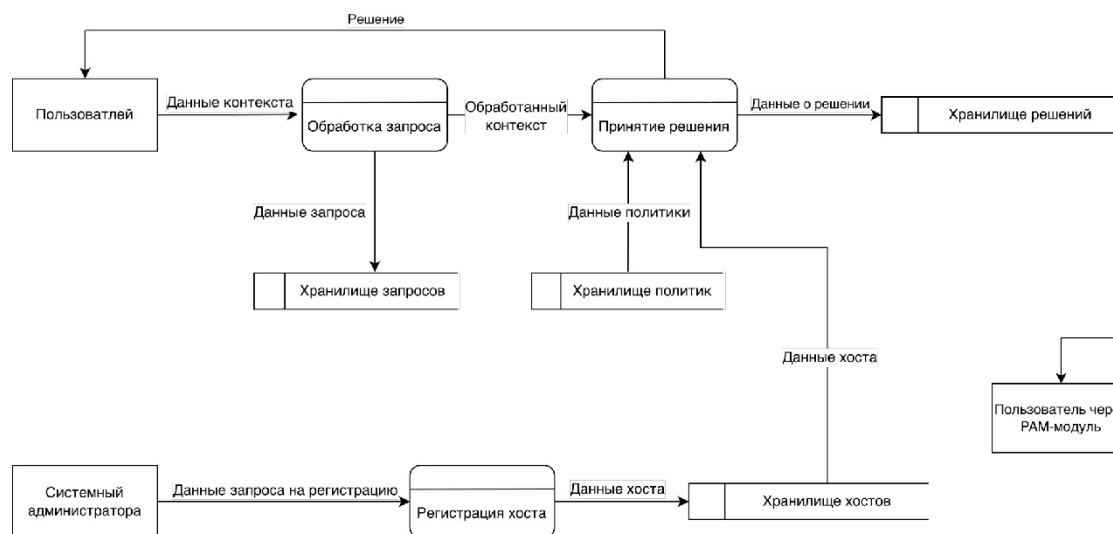
Основным функционалом системы является принятие решения о предоставлении доступа



Проектирование потоков и схемы данных

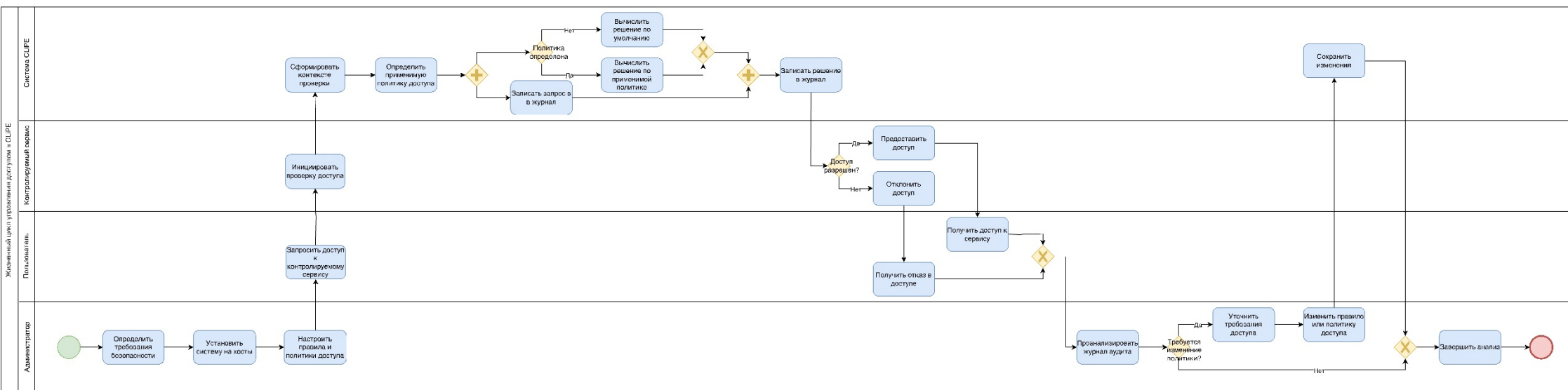


Описание потоков данных



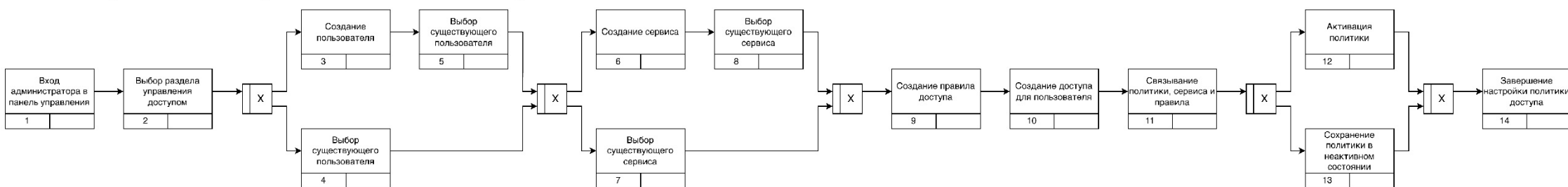
Проектирование бизнес-процесса

Основным бизнес процессом системы является управление доступом для хостов

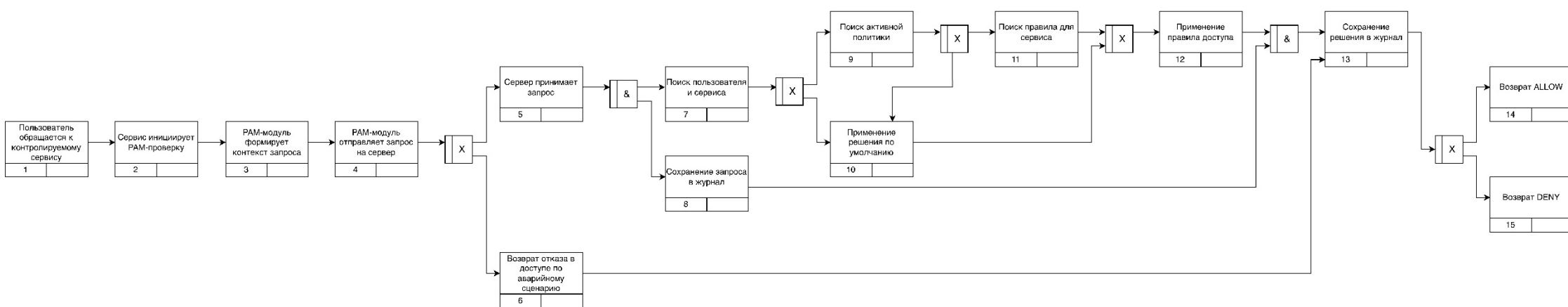


Проектирование сценариев взаимодействия

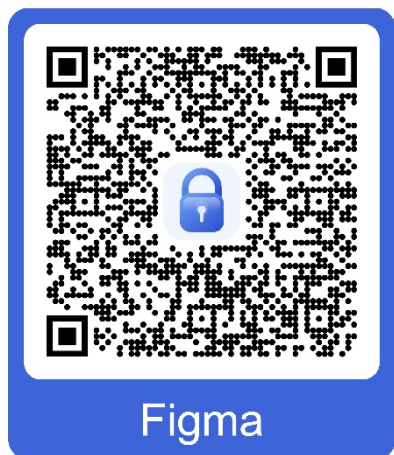
Процесс настройки политик доступа



Процесс получения решения о предоставлении доступа



Прототип

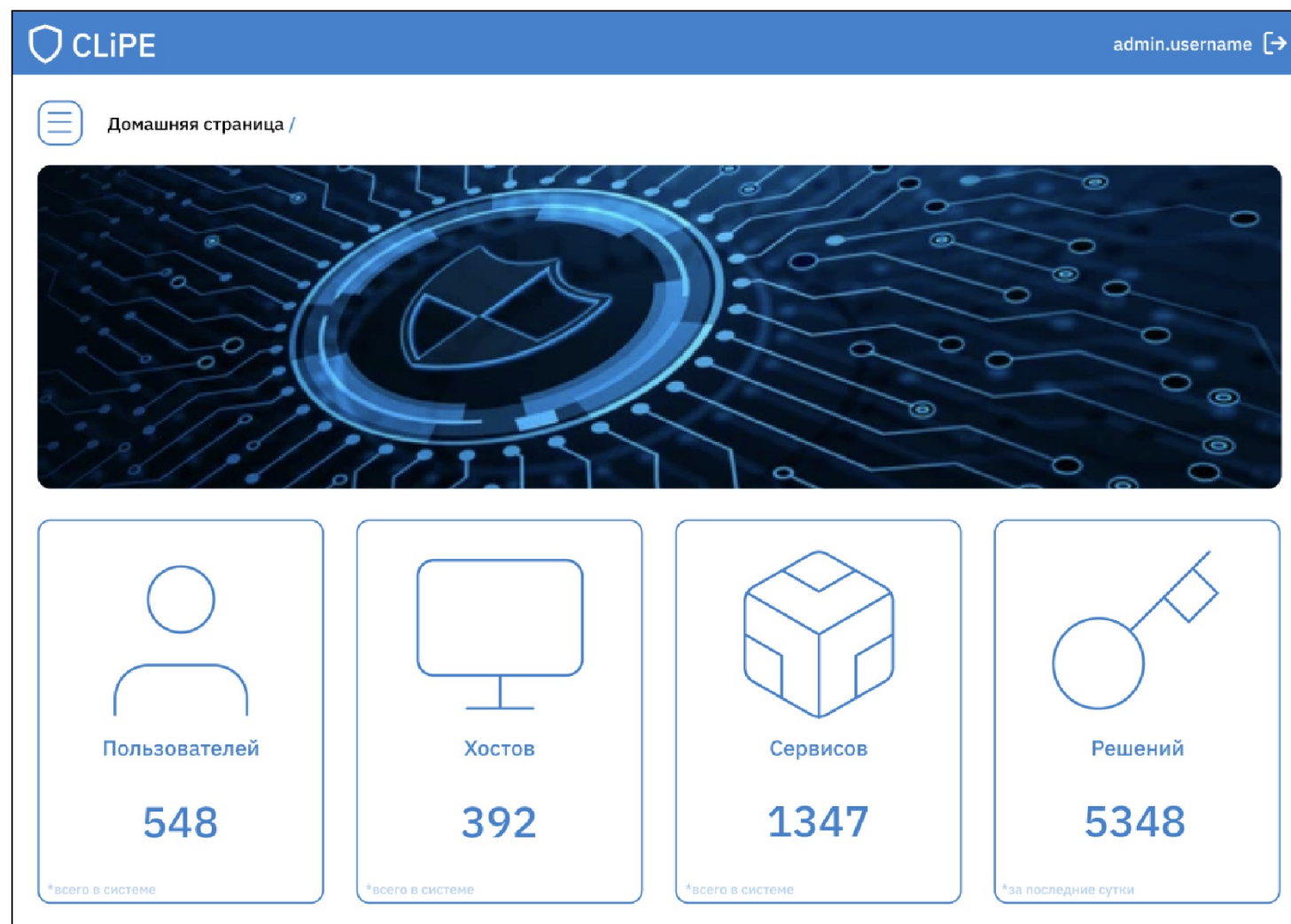


 CLIPE

Авторизация в системе

Войти

Прототип

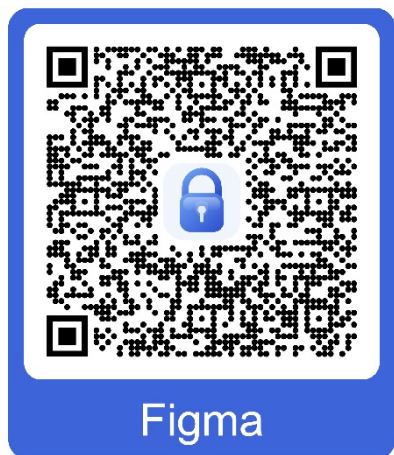


Прототип



CLiPE						admin.username [→]
Аудит / Журнал запросов						
Пользователь ▾		Хост ▾		Сервис ▾	Дата и время ▾	Показать
user.manager	@	192.168.232.123	→	ssh	at 21:57 16.03.2026	
user.analysts	@	192.168.232.123	→	login	at 21:57 16.03.2026	
user.	@	192.168.232.123	→	su	at 21:57 16.03.2026	
user.developer	@	192.168.232.123	→	sshd	at 21:57 16.03.2026	
user.admin	@	192.168.232.123	→	sudo	at 21:57 16.03.2026	
user.manager	@	192.168.232.123	→	ssh	at 21:57 16.03.2026	
user.manager	@	192.168.232.123	→	ssh	at 21:57 16.03.2026	
user.manager	@	192.168.232.123	→	ssh	at 21:57 16.03.2026	
user.manager	@	192.168.232.123	→	ssh	at 21:57 16.03.2026	

Прототип



 CLiPE

admin.username 

 Аудит / Журнал запросов / XXXXXXXX-XXXX-XXXX-XXXX-XXXXXXXXXXXX

Пользователь: user.name
IP-адрес хоста: 192.168.1.10
Сервис: ssh
Дата и время: 19:00 16.03.2026

JSON запрос

```
{
  "subject": {
    "username": "username",
    "uid": 1001,
    "gid": 1001,
    "groups": ["sudo", "dev"]
  },
  "resource": {
    "service": "ssh",
    "host": "server-1",
    "path": "/etc/passwd"
  },
  "action": "read",
  "context": {
    "timestamp": "2026-03-16T19:00:00Z",
    "ip": "192.168.1.10"
  }
}
```

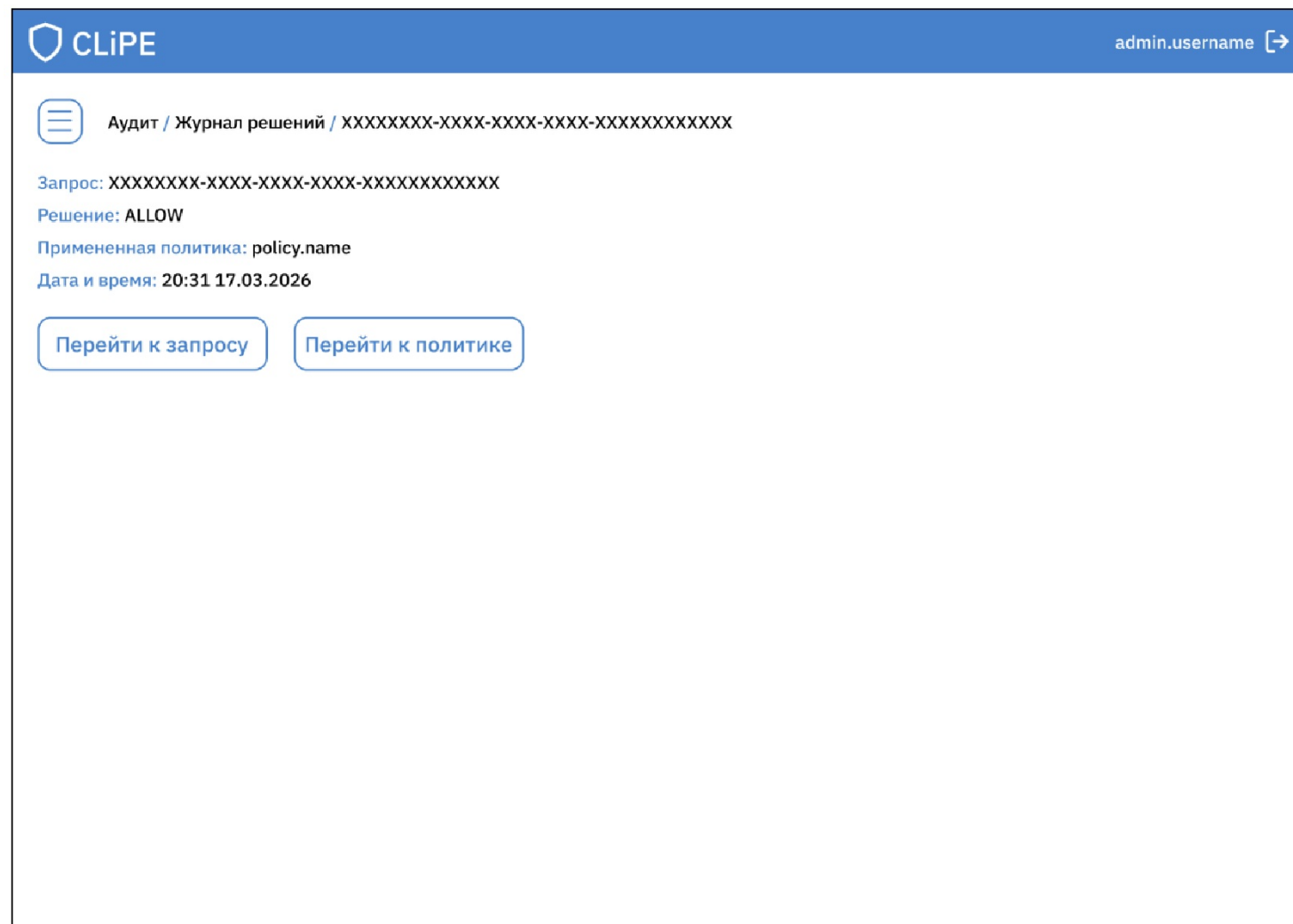
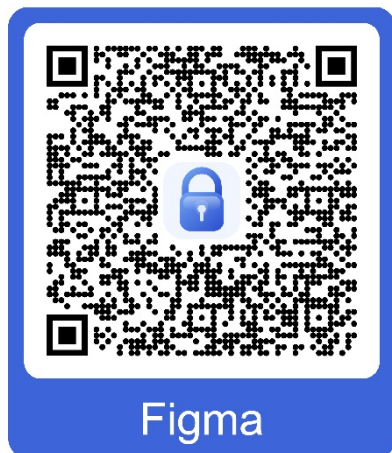
Перейти к решению

Прототип



CLiPE					admin.username [→]
Аудит / Журнал решений					
Запрос	Решение	Дата	Показать		
XXXXXXXX-XXXX-XXXX-XXXX-XXXXXXXXXXXX	ALLOW	at 19:56 17.03.2026			
XXXXXXXX-XXXX-XXXX-XXXX-XXXXXXXXXXXX	ALLOW	at 19:56 17.03.2026			
XXXXXXXX-XXXX-XXXX-XXXX-XXXXXXXXXXXX	ALLOW	at 19:56 17.03.2026			
XXXXXXXX-XXXX-XXXX-XXXX-XXXXXXXXXXXX	ALLOW	at 19:56 17.03.2026			
XXXXXXXX-XXXX-XXXX-XXXX-XXXXXXXXXXXX	ALLOW	at 19:56 17.03.2026			
XXXXXXXX-XXXX-XXXX-XXXX-XXXXXXXXXXXX	ALLOW	at 19:56 17.03.2026			
XXXXXXXX-XXXX-XXXX-XXXX-XXXXXXXXXXXX	ALLOW	at 19:56 17.03.2026			
XXXXXXXX-XXXX-XXXX-XXXX-XXXXXXXXXXXX	ALLOW	at 19:56 17.03.2026			
XXXXXXXX-XXXX-XXXX-XXXX-XXXXXXXXXXXX	ALLOW	at 19:56 17.03.2026			

Прототип



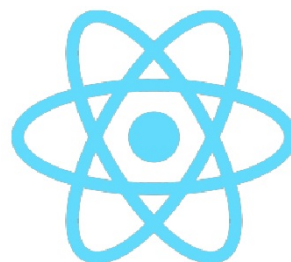
Используемые технологии



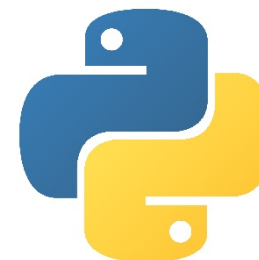
API и сервер
принятия решений



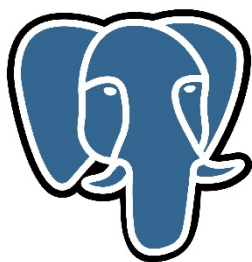
РАМ модуль



Веб-интерфейс



Вспомогательные скрипты



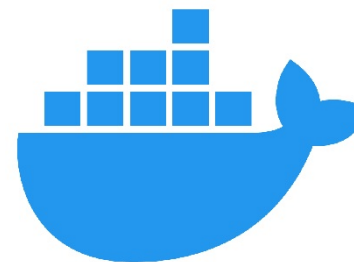
База данных



Сборка C++

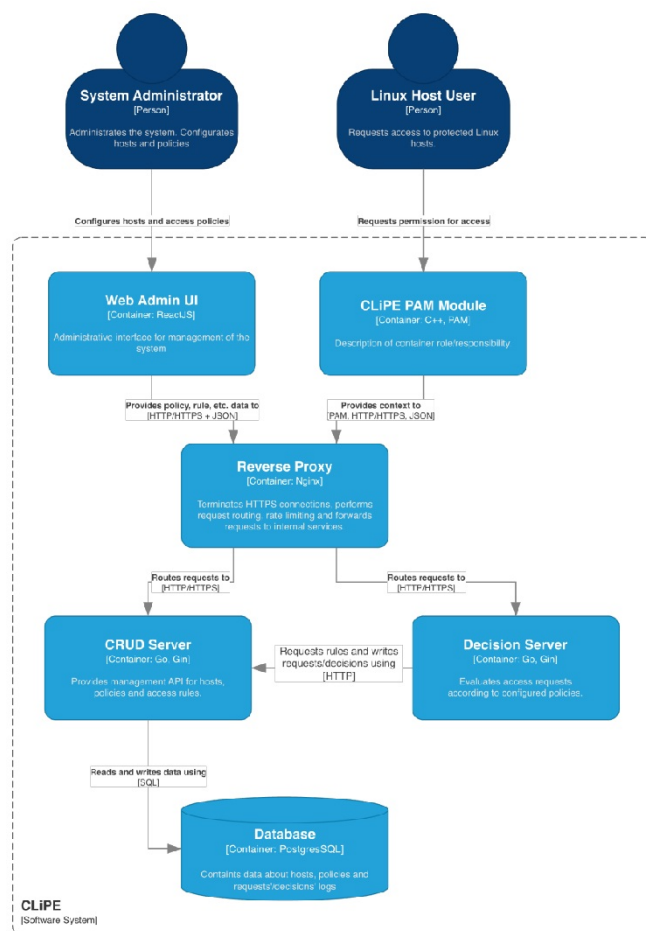


Reverse Proxy



Контейнеризация

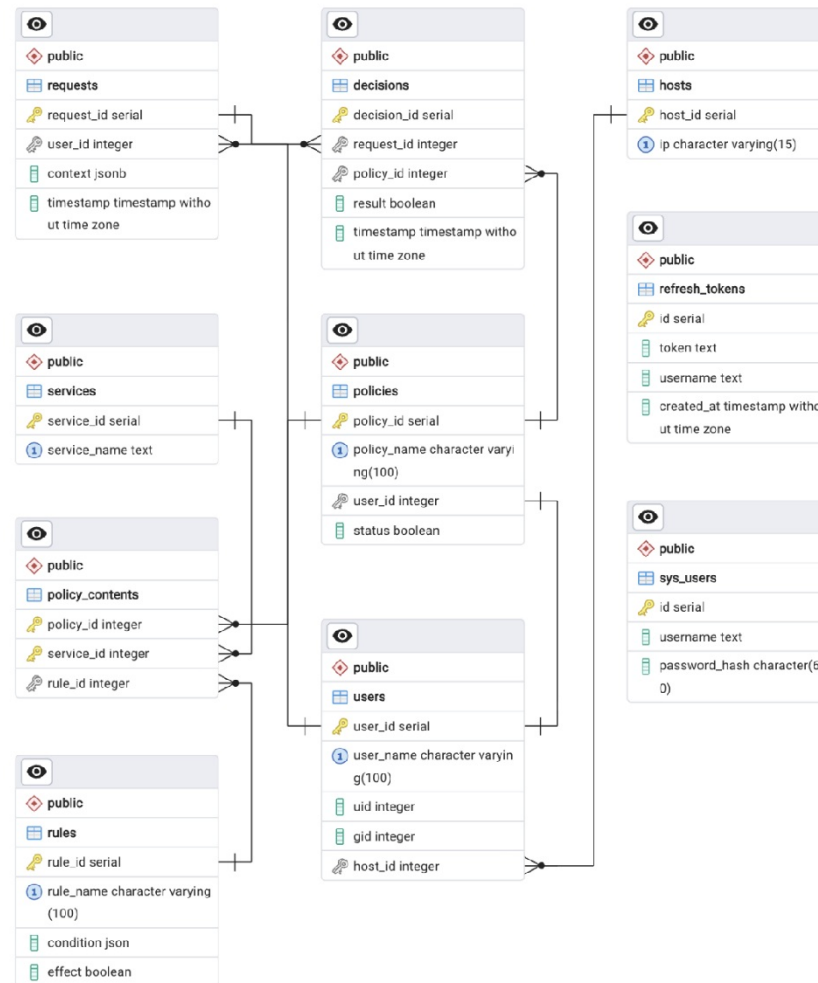
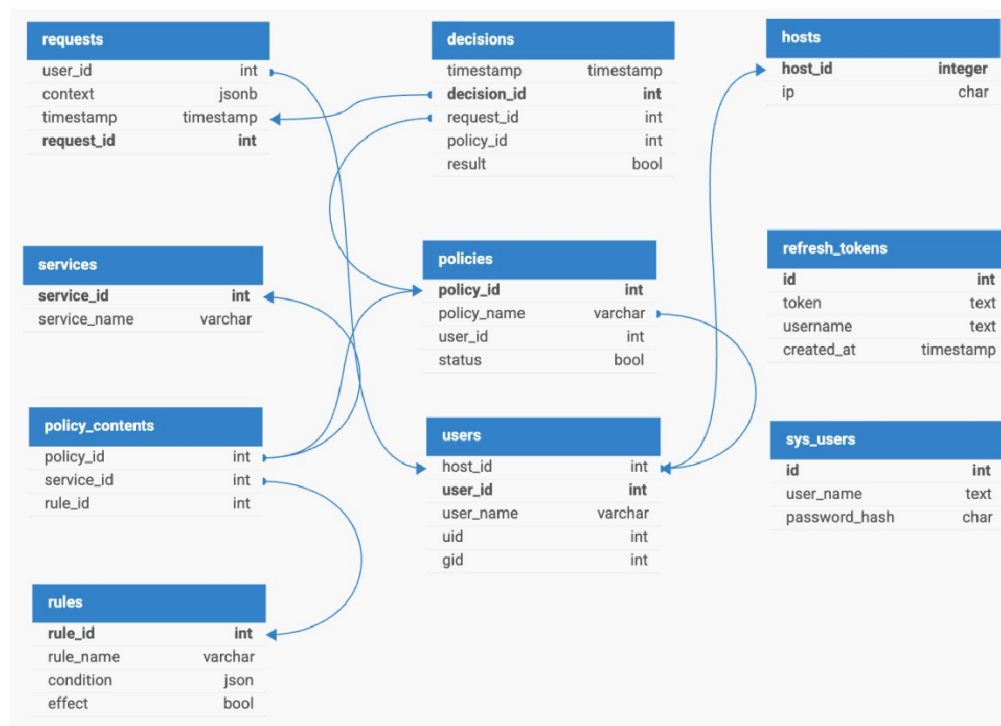
Построение архитектуры



- Передача информации о политиках и правилах
- Запрос доступа к сервису
- Передача данных в формате JSON
- Передача контекста запроса
- Перенаправление запроса к соответствующему серверу
- Запись/чтение данных о политиках, правилах и хостах
- Получение правил и журналирование запросов/решений

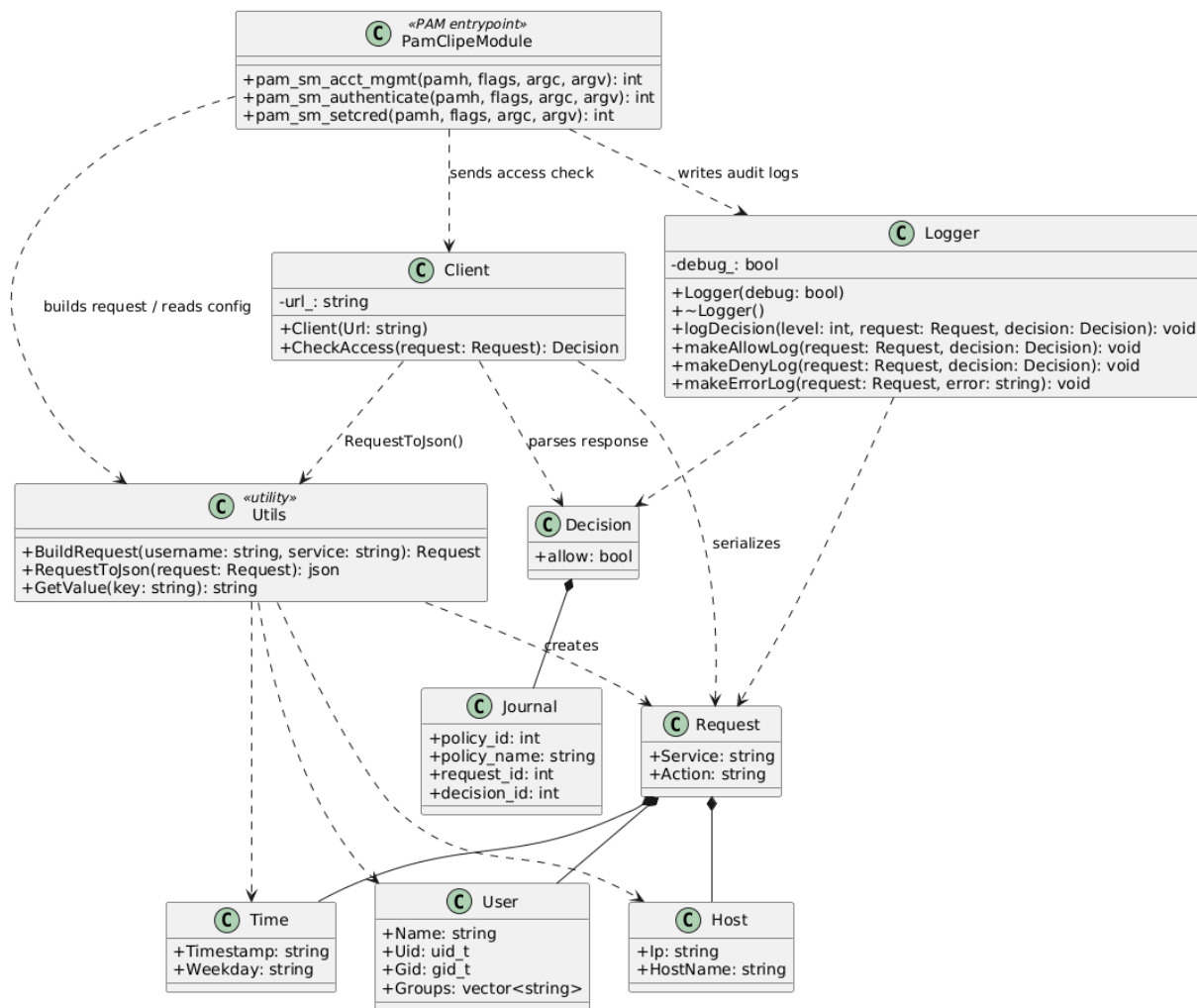
Проектирование базы данных

Под требуемый функционал, схему данных и архитектуру была спроектирована и реализована реляционная база данных



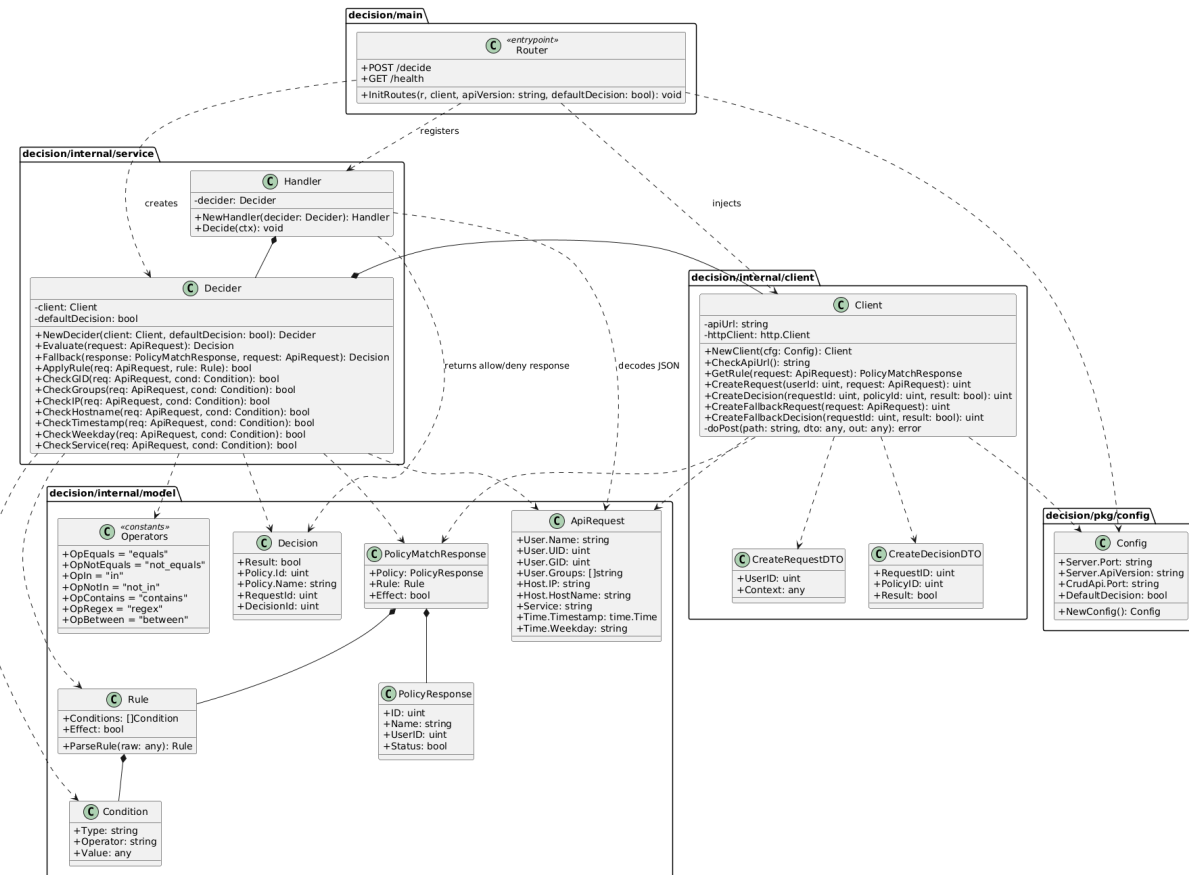
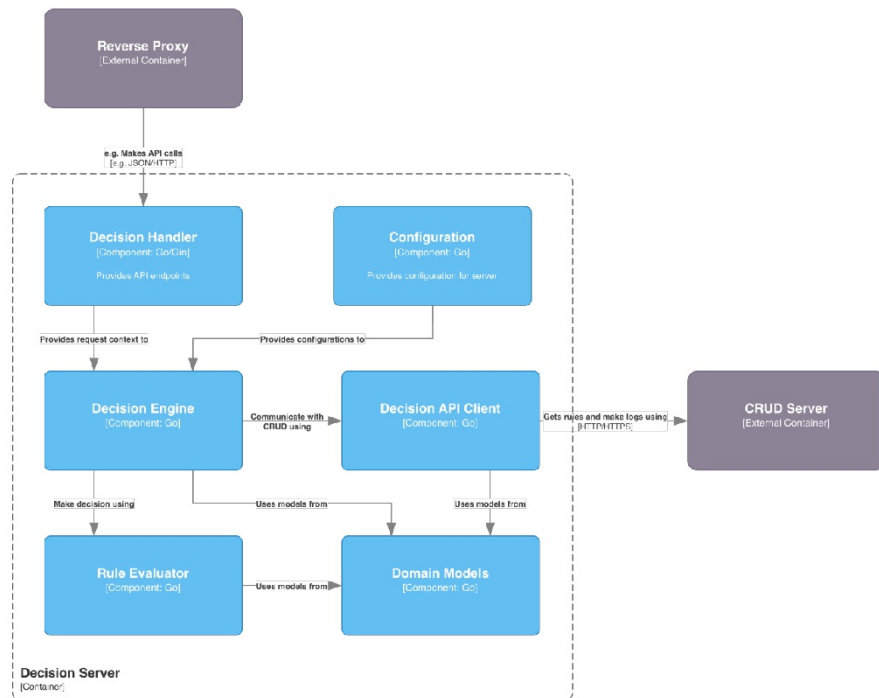
Реализация

Разработана динамическая библиотека для встраивания в нативные PAM-таблицы, выступающая в роли клиента системы



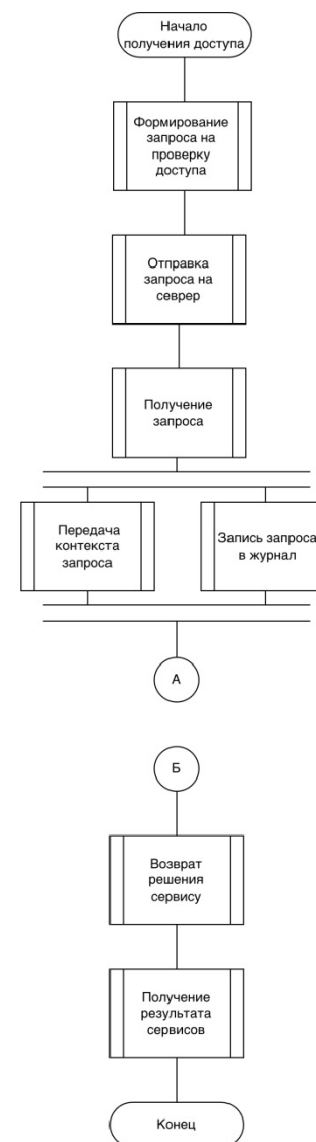
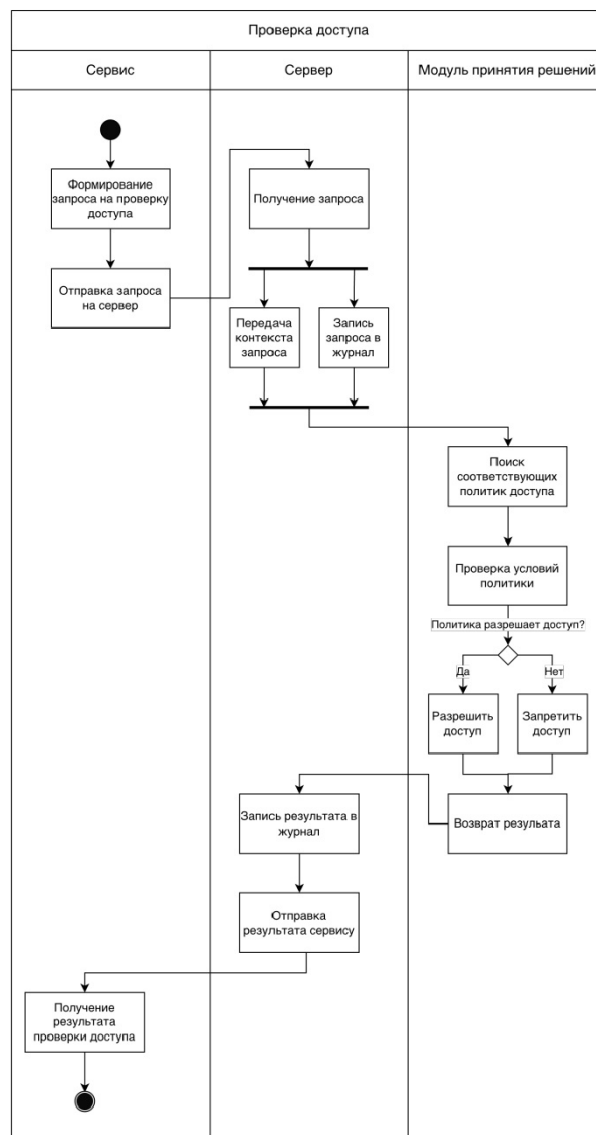
Реализация

Разработан сервер принятия решений,
являющийся основным компонентом системы



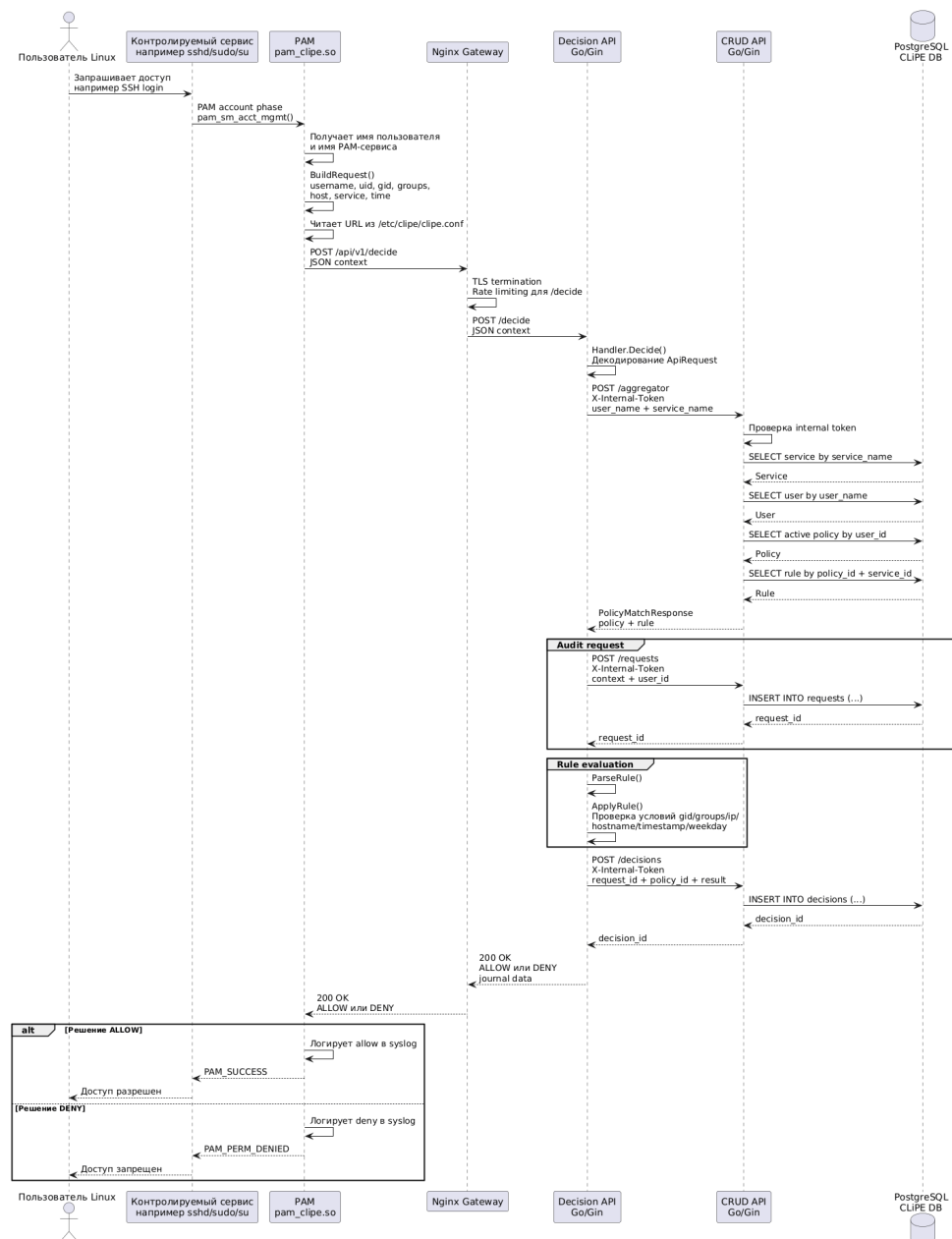
Реализация

Спроектирован алгоритм
принятия решения о
предоставлении доступа на
основе контекста,
формируемого РАМ-модулем

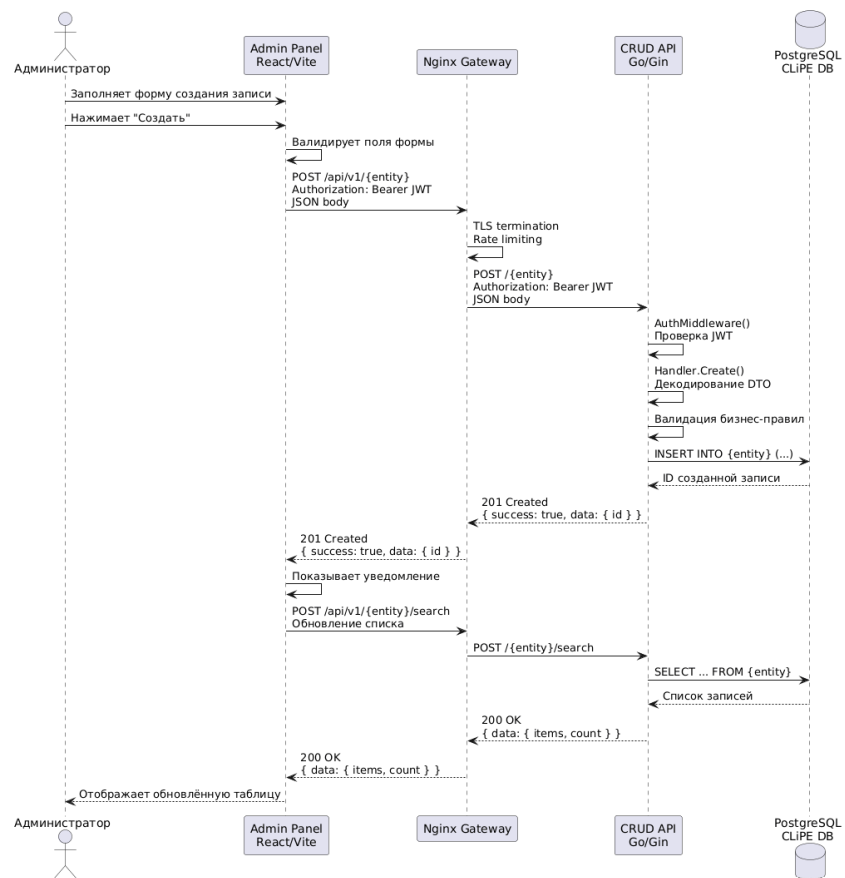


Работа системы

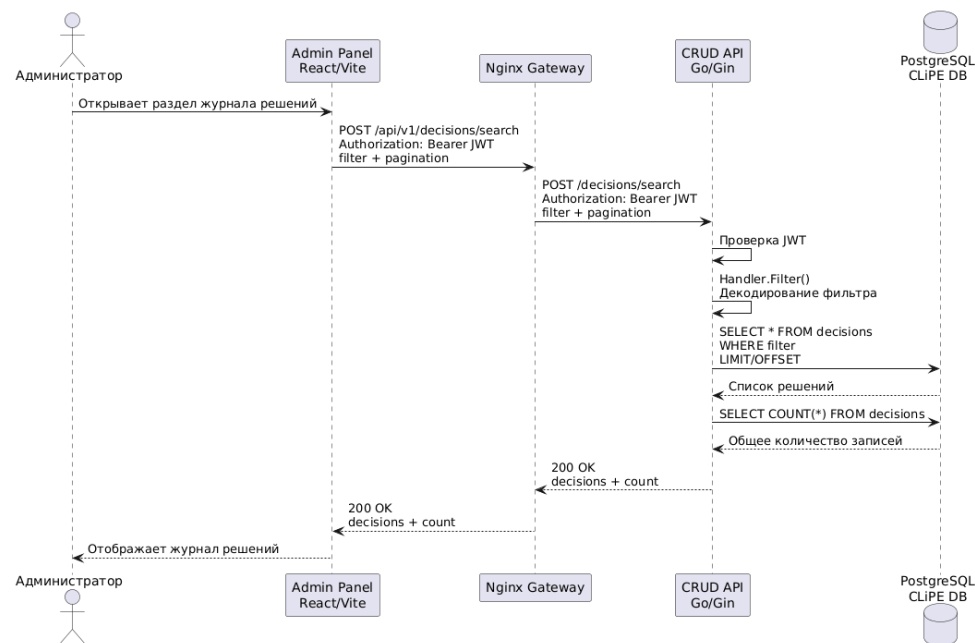
Диаграмма последовательности демонстрирует процесс получения доступа к контролируемому сервису и порядок взаимодействия между PAM-модулем, сервером принятия решений и базой данных.



Работа системы



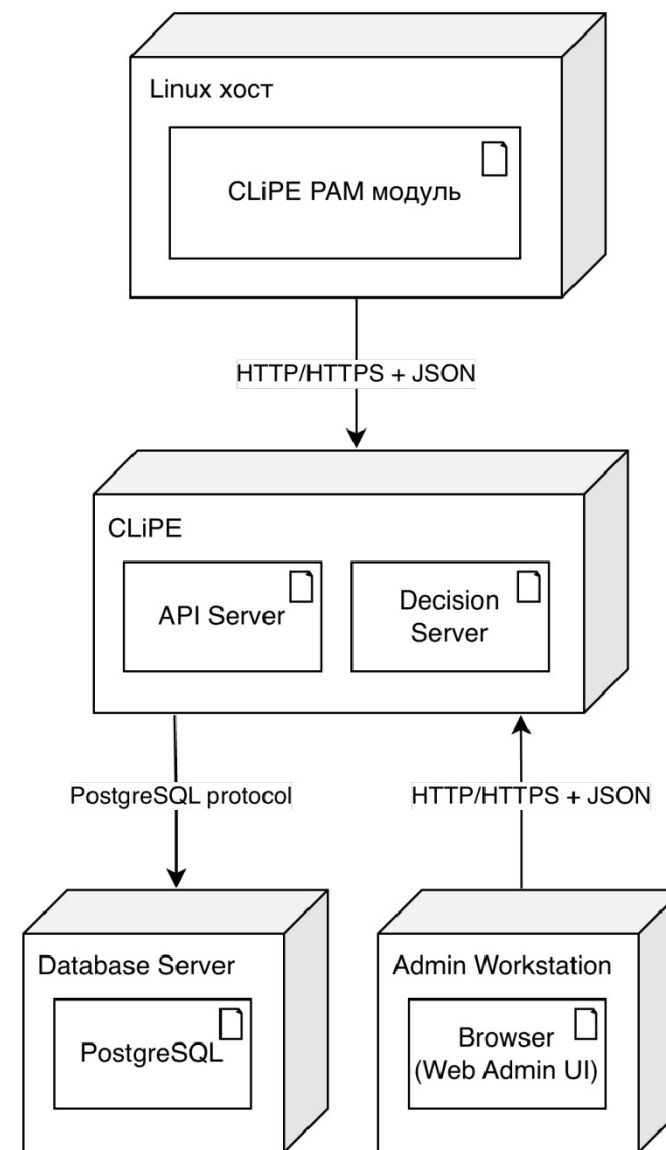
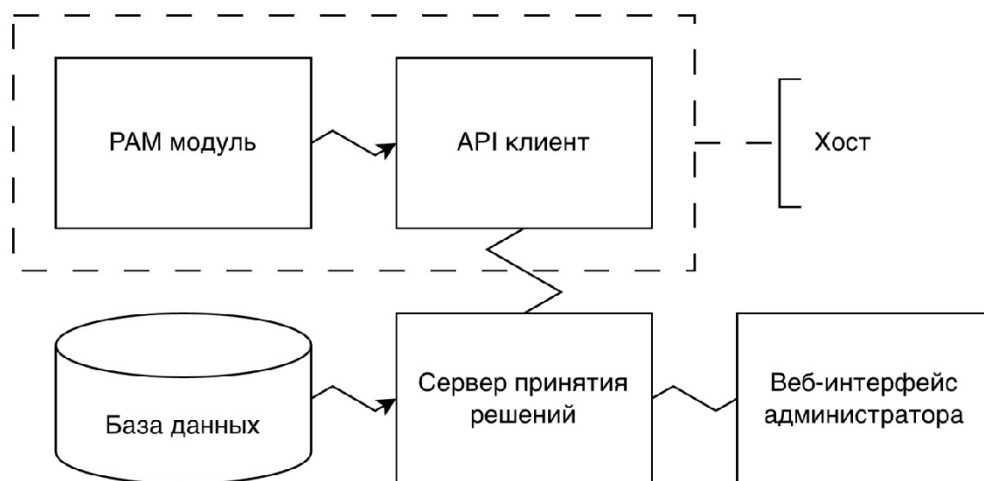
Создание записи через веб-интерфейс



Просмотр страница
(на примере журнала)

Развертывание

Диаграмма развертывания демонстрирует распределение компонентов системы между вычислительными узлами
Исходный код и инструкция по установке доступным на GitHub



Тестирование

Выполнено:

- 14 тест-кейсов с 42 наборами тестовых данных (веб-интерфейс, РАМ-модуль)
- 11 тестовых коллекций для тестирования API с 93 тестами (CRUD сервер и E2E тестирование)
- 10 тестовых методов с 61 набором тестовым данных (Decision Engine)

Тест-кейсы

ID тест-кейса/Приоритет	TU03	3
Идея: Проверка корректного функционирования логики “Решение по умолчанию”		
Входные параметры: —		
История редактирования		
Создан 11.05.2026	Новый тест-кейс	
Предусловие		
DEFAULT_DECISION=true		
URL в /etc/clipe/clipe.conf корректный		
DNS сервера принятий решений в /etc/hosts верный		
Политика для данного пользователя НЕ существует		
Выполняемая часть		
Список команд	Ожидаемый результат	Результат теста
1)python3 test.py	Вывод сообщение “Access granted” В логах: - result=ALLOW - policy_name=unknown	Pass
2)tail -n 1 /var/log/auth.log		

ID тест-кейса/Приоритет	TU14	14
Идея: Проверка корректности навигации по страницам интерфейса		
Входные параметры:		
- логин: admin		
- пароль: admin		
История редактирования		
Создан 11.05.2026	Новый тест-кейс	
Предусловие		
Открытая страница входа в систему		
Выполняемая часть		
Список команд	Ожидаемый результат	Результат теста
1)Ввести входные данные на странице входа	1)-	Pass
2)Нажать “Войти”	2)Переход на домашнюю сраницу	
3)Нажать “Перейти к решениям”	3)Переход на страницу решений	
4)Нажать “Запрос №N”	4)Переход на страницу запросов	
5)Нажать “Перейти к политике”	5)Переход на страницу привязки правил для данной политики	
6)Нажать на название любого правила	6)Переход на страницу данного правила	
7)Выбрать в боковом меню “Домашняя страниц”	7)Переход на домашнюю страницу	
8)Нажать “Сервисы”	8)Переход на страницу сервисов	
9)Выбрать в боковом меню “Хосты”	9)Переход на страницу хостов	
10)Выбрать в боковом меню “Политики”	10)Переход на страницу политик	
11)Нажать на имя любого пользователя	11)Переход на страницу данного пользователя	
12)Нажать “Выйти” в хедере страницы	12)Переход на страницу входа	

Тест-кейсы

ID тест-кейса/Приоритет	TU07	7
Идея: Проверка корректности работы фильтра при различных параметрах фильтров		
Входные параметры: входные параметры указаны в Таблице 7.1		
История редактирования		
Создан 11.05.2026	Новый тест-кейс	
Предусловие		
Открыта страница “Пользователи”		
Выполняемая часть		
Список команд	Ожидаемый результат	Результат теста
1)Ввести параметры фильтра из входных данных 2)(Опционально) Нажать “Сбросить фильтры”	В соответствии с входными данными представлен в Таблице 7.1	Pass

№ п/п	Параметры фильтров	Ожидаемый результат
1	-	Вывод записей в соответствии с выбранной страницей списка
2	Данные несуществующей записи	Вывод сообщение “По текущим фильтрам записи не найдены”
3	' OR '1'='1'; DROP DATABASE; --	Вывод сообщение “По текущим фильтрам записи не найдены”. База данных не пострадала
4	<script>alert(1);</script>	Вывод сообщение “По текущим фильтрам записи не найдены”. Окно предупреждения не открылось
5	Данные существующей записи	Вывод записей соответствующих заданному фильтру

Модульное тестирование

- ✓ TestNewDecider decider_test.go < test < decision
- ✓ TestFallback decider_test.go < test < decision
- ✓ TestEvaluate decider_test.go < test < decision
- ✓ TestApplyRule decider_test.go < test < decision
- ✓ TestCheckGID decider_test.go < test < decision
- ✓ TestCheckGroups decider_test.go < test < decision
- ✓ TestCheckIP decider_test.go < test < decision
- ✓ TestCheckHostname decider_test.go < test < decision
- ✓ TestCheckTimestamp decider_test.go < test < decision
- ✓ TestCheckWeekday decider_test.go < test < decision
- ✓ TestCheckService decider_test.go < test < decision

```
t.Run("success uses default decision and unknown policy", func(t *testing.T) {
    client := &fakeDecisionClient{
        createFallbackRequestID: 10,
        createFallbackDecisionID: 20,
    }
    decider := service.NewDecider(client, true)

    got, err := decider.Fallback(nil, req)

    assert.NoError(t, err)
    assert.Equal(t, true, got.Result)
    assert.Equal(t, uint(0), got.Policy.Id)
    assert.Equal(t, "unknown", got.Policy.Name)
    assert.Equal(t, uint(10), got.RequestId)
    assert.Equal(t, uint(20), got.DecisionId)
})
```

run test | debug test

```
t.Run("create fallback request error", func(t *testing.T) {
    decider := service.NewDecider(&fakeDecisionClient{
        createFallbackRequestErr: errors.New("request failed"),
    }, false)

    got, err := decider.Fallback(nil, req)

    assert.Nil(t, got)
    assert.ErrorContains(t, err, "error in CreateRequest")
})
```

run test | debug test

```
t.Run("create fallback decision error", func(t *testing.T) {
    decider := service.NewDecider(&fakeDecisionClient{
        createFallbackRequestID: 10,
        createFallbackDecisionErr: errors.New("decision failed"),
    }, false)

    got, err := decider.Fallback(nil, req)

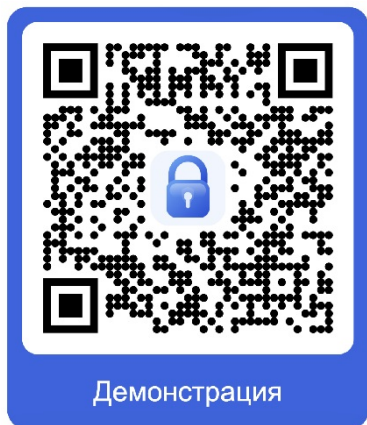
    assert.Nil(t, got)
    assert.ErrorContains(t, err, "error in CreateDecision")
})
```

Тестирование API

```
1 // Фрагмент кода e2e тестирования (post-script для call_pdp)
2 pw.test("result exists", () => {
3   pw.expect(pw.response.body.result).toBe("ALLOW")
4 })
5
6 pw.test("decision exists", () => {
7   pw.expect(pw.response.body.journal.decision_id).toBeType("number")
8   pw.env.set("e2e_decision_id", String(pw.response.body.journal.decision_id))
9 })
10
11 pw.test("request exists", () => {
12   pw.expect(pw.response.body.journal.request_id).toBeType("number")
13   pw.env.set("e2e_request_id", String(pw.response.body.journal.request_id))
14 })
15
16
17 // Фрагмент кода e2e тестирования (post-script для check_request)
18 pw.test("check user_id", () => {
19   pw.expect(pw.response.body.data.requests[0].user_id).toBe(Number(pw.env.get("e2e_user_id")))
20 })
21
22 pw.test("check request_id", () => {
23   pw.expect(pw.response.body.data.requests[0].request_id).not.toBe(undefined)
24   pw.env.set("e2e_request_id", String(pw.response.body.data.requests[0].request_id))
25 })
```

clipe		+	
Collection	Environment	Duration	Avg. Response Time
clipe	CLiPE	2s 327ms	25ms
All Tests	Passed	Failed	
132	132	0	

Демонстрация работы системы



Спасибо за внимание!



Figma



GitHub



Документация



Демонстрация